Representação de informação

Formatos e operações

João Canas Ferreira

Setembro de 2026

U.PORTO
FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

Tópicos

1. Introdução à representação de informação
2. Representação em binário de números sem sinal
   - Representação binária
   - Conversão de base
   - O sistema de numeração hexadecimal
   - Operações aritméticas com operandos em binário
3. Representação de números inteiros com sinal
   - Operandos e resultado de tamanho fixo
   - Sinal e grandeza
   - Representação em complemento para 2
4. Representação de carateres e símbolos gráficos
5. Representação de números em vírgula flutuante
   - Conceitos gerais
   - Formato IEEE-754 precisão simples
   - Operações aritméticas com operandos normalizados

1 Introdução à representação de informação

2 Representação em binário de números sem sinal
- Representação binária
- Conversão de base
- O sistema de numeração hexadecimal
- Operações aritméticas com operandos em binário

3 Representação de números inteiros com sinal
- Operandos e resultado de tamanho fixo
- Sinal e grandeza
- Representação em complemento para 2

4 Representação de carateres e símbolos gráficos

5 Representação de números em vírgula flutuante
- Conceitos gerais
- Formato IEEE-754 precisão simples
- Operações aritméticas com operandos normalizados

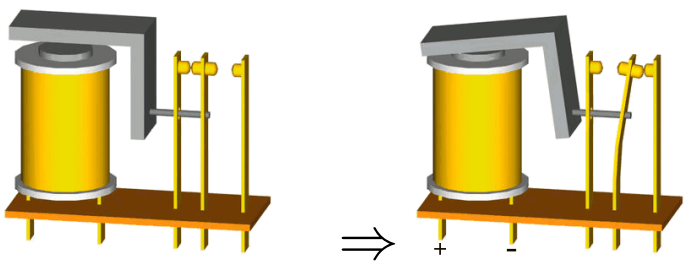
Representação física da informação

➠ O computador `automático` pode ser definido como uma máquina de uso geral que processa informação de acordo com uma sequência de instruções.

Questão: Como representar fisicamente a informação?

➠ As técnicas de implementação recorrem a elementos básicos que podem estar em um de dois estados (elementos bi-estáveis).

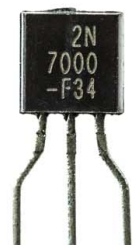
Relé (eletromecânico)



Tubo de vácuo (válvula)



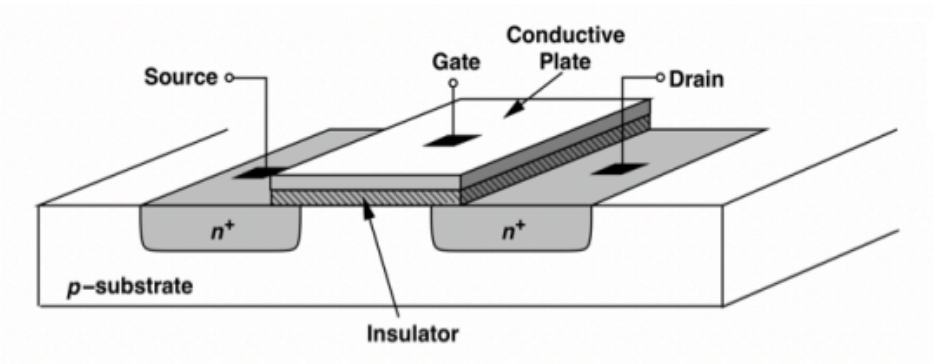
Transístor discreto

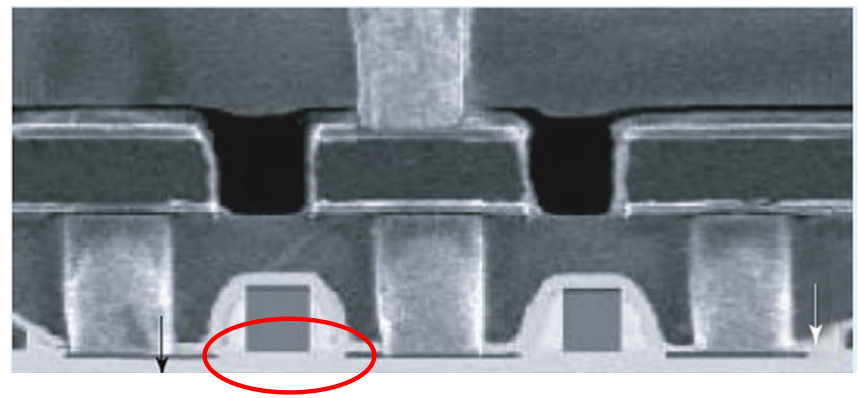


Transístores integrados

➠ Os elementos ativos dos processadores atuais são dispositivos eletrónicos (transístores) que atuam como interrutores que podem estar **fechados**  ou **abertos** .

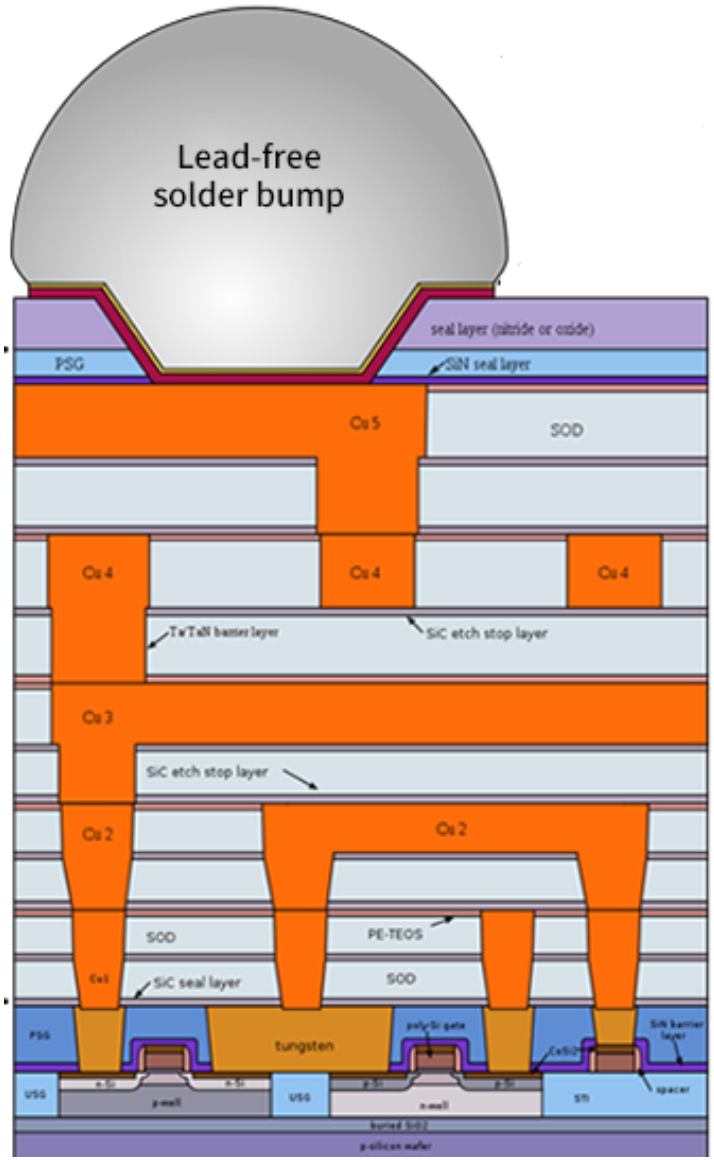
Os transístores são feitos de silício (um material semicondutor) e vários compostos de silício, incluindo óxido de silício, um dos materiais mais comuns na Terra (constituinte principal da areia).

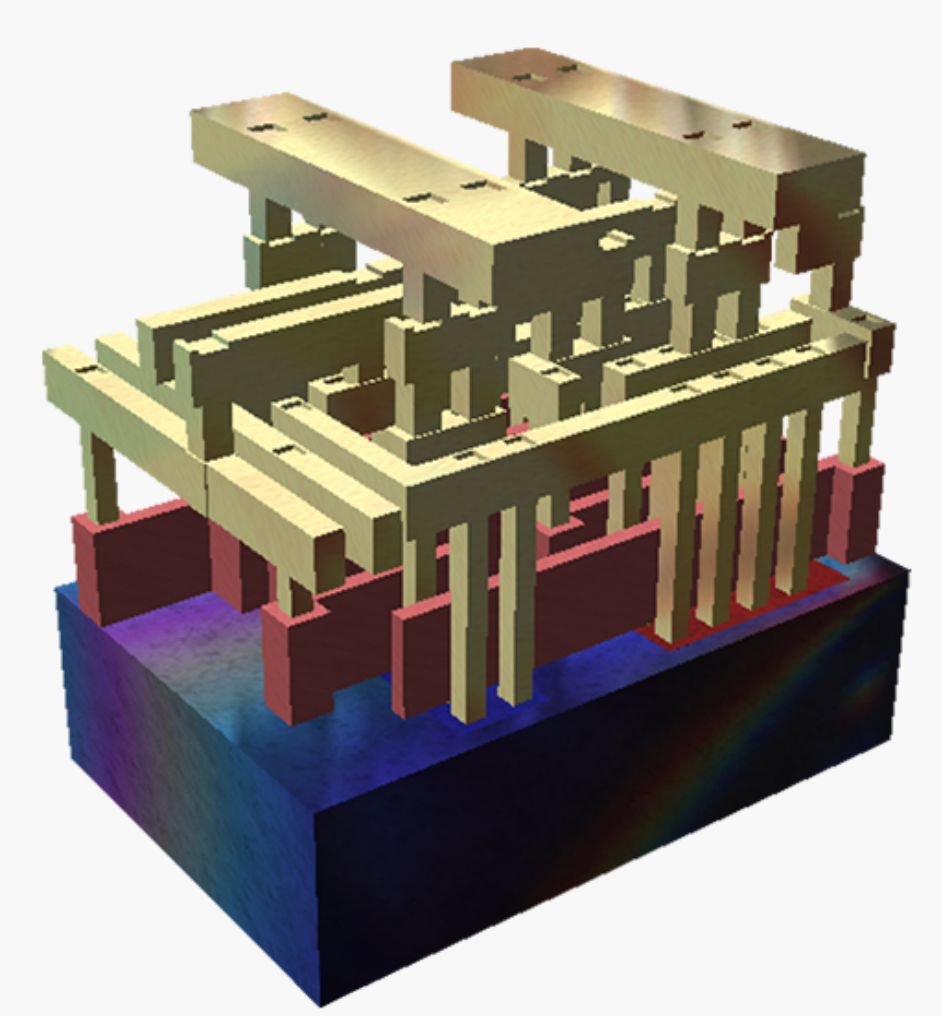




➠ Os microprocessadores atuais são **circuitos integrados** que contém milhares de milhões (10^9) de transístores, que são capazes de mudar de estado em **algumas dezenas de picossegundos** (1 ps = 1×10^{-12} s).

Interior de um circuito integrado: transístores e interligações





Representação simbólica da informação

- Para termos uma **representação abstrata**, desligada dos detalhes da realização física, os estados são representados pelos símbolos **0** e **1**.
- A unidade fundamental de informação designa-se por **bit**.
- O estado de um interrutor representa fisicamente 1 bit de informação.
- Para representar maior quantidade de informação usam-se conjuntos (ordenados) de bits. Exemplo: o grupo «1011» tem 4 bits $b_3\ b_2\ b_1\ b_0$.

bits	**1**	**0**	**1**	**1**
posição	**3**	**2**	**1**	**0**

Neste caso, $b_2 = 0$ e $b_0 = 1$

  As posições são numeradas a partir de **zero** da direita para a esquerda.
- Um grupo de 8 bits designa-se por **octeto**.
- Os processadores tratam grupos de bits: o grupo mais pequeno usado por um processador designa-se por **byte** (unidade: B).
- Computadores atuais: 1 B = 8 bit (um byte é o mesmo que um octeto).

1. Introdução à representação de informação
2. Representação em binário de números sem sinal
   - Representação binária
   - Conversão de base
   - O sistema de numeração hexadecimal
   - Operações aritméticas com operandos em binário
3. Representação de números inteiros com sinal
   - Operandos e resultado de tamanho fixo
   - Sinal e grandeza
   - Representação em complemento para 2
4. Representação de carateres e símbolos gráficos
5. Representação de números em vírgula flutuante
   - Conceitos gerais
   - Formato IEEE-754 precisão simples
   - Operações aritméticas com operandos normalizados

Sistema de numeração decimal

- O sistema de numeração decimal é caraterizado por:
  - ☞ usar como base o valor 10;
  - ☞ usar 10 dígitos (símbolos individuais): 0, 1,..., 9.

- Uma sequência de N dígitos (um número) representa a soma do produto de que cada dígito por 10^p, em que p indica a posição do dígito no número.
  Por exemplo, a sequência de dígitos «4542» representa o valor

$$4 \times 10^3 + 5 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$$

  ➠ 4 milhares (10^3), 5 centenas (10^2), 4 dezenas (10^1) e 2 unidades (10^0).

- Nestes sistemas, o peso de cada dígito é determinado pela sua posição.
  No número «4542», as duas instâncias do símbolo «4» têm pesos diferentes (10^3 e 10^1, respetivamente).
  Trata-se de um sistema de numeração **posicional**.

- Exemplo de um sistema não posicional: numeração romana.

Sistema de numeração binário

- Num computador existem apenas dois símbolos que podem servir de dígitos (0 e 1). Logo, temos que usar base $b = 2$ para representar números.
- Um conjunto de N bits pode ser interpretado como um número em base 2, i.e., uma soma de potências de 2:

$$b_{N-1} \times 2^{N-1} + b_{N-2} \times 2^{N-2} + \ldots + b_1 \times 2^1 + b_0 \times 2^0$$

  Exemplo: $10010_2 = 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2 + 0 = 16 + 2 = 18_{10}$
- Consequência: todos os números **pares** têm uma representação binária terminada por 0.
- Consequência: as regras para as operações aritméticas em binário **são análogas às regras usadas com o sistema decimal**, porque dependem apenas das propriedades dos sistemas de numeração posicionais.
- A conversão binário→decimal pode ser feita aplicando diretamente a definição e realizando os cálculos em decimal (como no exemplo acima).

Representação de parte fracionária

- Um sistema posicional pode também representar números com parte fracionária usando potências inteiras **negativas** da base ($b^{-1}, b^{-2}, \ldots$)
- A vírgula é usada para separar as partes inteira e fracionária.

  Exemplo em decimal: $17{,}83_{10} = 1 \times 10^1 + 7 + 8 \times \frac{1}{10} + 3 \times \frac{1}{10^2}$

  Em palavras: 1 dezena, 7 unidades, 8 décimas e 3 centésimas.
- Exemplo em binário:

  $$101{,}101_2 = 1 \times 2^2 + 1 + 1 \times \frac{1}{2} + 1 \times \frac{1}{2^3} = 4 + 1 + 0{,}5 + 0{,}125 = 5{,}625_{10}$$
- Em qualquer sistema posicional, pode considerar-se que existe um número infinito de zeros à esquerda e à direita da representação dada.

  Exemplos: $0076{,}1400_{10} = 76{,}14_{10}$ $00101{,}1100_2 = 101{,}11_2$.

  Convencionalmente, esses zeros não são representados.

1. Introdução à representação de informação
2. Representação em binário de números sem sinal
   - Representação binária
   - Conversão de base
   - O sistema de numeração hexadecimal
   - Operações aritméticas com operandos em binário
3. Representação de números inteiros com sinal
   - Operandos e resultado de tamanho fixo
   - Sinal e grandeza
   - Representação em complemento para 2
4. Representação de carateres e símbolos gráficos
5. Representação de números em vírgula flutuante
   - Conceitos gerais
   - Formato IEEE-754 precisão simples
   - Operações aritméticas com operandos normalizados

Conversão binário $\rightarrow$ decimal

- Um número em binário corresponde a uma soma de potências inteiras de 2. Para converter um número binário em decimal basta executar as operações implícitas na representação posicional usando base 10.

Exemplo de conversão base 2 $\rightarrow$ base 10

$$1101{,}011_2 = 2^3 + 2^2 + 2^0 + 2^{-2} + 2^{-3} = 8 + 4 + 1 + 0{,}25 + 0{,}125 = 13{,}375_{10}$$

- A posição imediatamente *à esquerda* da vírgula corresponde ao expoente 0.
- A posição imediatamente *à direita* da vírgula corresponde ao expoente -1.
- Expoentes crescem para a esquerda da vírgula, decrescem para a direita.
- Não é preciso incluir nos cálculos as potências cujo coeficiente é zero.

Conversão decimal $\rightarrow$ binário: parte inteira

- Dividir o número sucessivamente por 2 até chegar a 0.
- Guardar o resto (0 ou 1) de cada divisão.
- Os valores sucessivos do resto constituem o número em binário, sendo que o primeiro resto corresponde ao bit *menos significativo* e o último ao bit *mais significativo*.

Exemplo de conversão base 10 $\rightarrow$ base 2

$$19_{10} =?_2$$

N	expressão	quociente	resto
19	$9 \times \mathbf{2} + 1$	9	1
9	$4 \times \mathbf{2} + 1$	4	1
4	$2 \times \mathbf{2} + 0$	2	0
2	$1 \times \mathbf{2} + 0$	1	0
1	$0 \times \mathbf{2} + 1$	0	1

$$\rightarrow 19_{10} = 10011_2$$

- Terminar quando o quociente atinge 0.

Conversão decimal $\rightarrow$ binário: parte fracionária

- Multiplicar o número por 2.
- Guardar a parte inteira (0 ou 1) de cada multiplicação.
- Os valores da parte inteira constituem a parte fracionária do número em binário. O primeiro valor é o bit *mais significativo* e o último ao *menos significativo*.
- Repetir o processo com a parte fracionária.
- Terminar: parte fracionária chega a 0 *ou* repete-se *ou* atinge um número dado de bits.

Exemplo de conversão base 10 $\rightarrow$ base 2: $0{,}6_{10}$

Número	$\times 2$	Parte inteira
0,6	1,2	1
0,2	0,4	0
0,4	0,8	0
0,8	1,60	1
0,6	…	…

$\rightarrow$

$0{,}6_{10} = 0{,}1001(1001)_2$
(parte fracionária infinita com repetição)

Valor (aproximado) com 8 casas binárias:
$0{,}10011001_2 = 2^{-1} + 2^{-4} + 2^{-5} + 2^{-8} = 0{,}59765625_{10}$

1. Introdução à representação de informação
2. Representação em binário de números sem sinal
   - Representação binária
   - Conversão de base
   - O sistema de numeração hexadecimal
   - Operações aritméticas com operandos em binário
3. Representação de números inteiros com sinal
   - Operandos e resultado de tamanho fixo
   - Sinal e grandeza
   - Representação em complemento para 2
4. Representação de carateres e símbolos gráficos
5. Representação de números em vírgula flutuante
   - Conceitos gerais
   - Formato IEEE-754 precisão simples
   - Operações aritméticas com operandos normalizados

Sistema de numeração com base 16

- O sistema *hexadecimal* usa base $b = 16$.
- O conjunto de dígitos é: `0`, `1`, `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`, `A`, `B`, `C`, `D`, `E`, `F`.
- Para os dígitos acima de 9, a correspondência com valores decimais é:

$$\text{A}\leftrightarrow 10 \quad \text{B}\leftrightarrow 11 \quad \text{C}\leftrightarrow 12 \quad \text{D}\leftrightarrow 13 \quad \text{E}\leftrightarrow 14 \quad \text{F}\leftrightarrow 15.$$

Conversão de hexadecimal para decimal

$6B7_{16} = 6B7_H = 6 \times 16^2 + 11 \times 16^1 + 7 = 1719_{10}$

$7F{,}5_{16} = 7 \times 16^1 + 15 \times 16^0 + 5 \times 16^{-1} = 127{,}3125_{10}$

- A conversão de decimal para hexadecimal usa o mesmo procedimento que a conversão para binário, mas dividindo ou multiplicando por 16 (em vez de 2).
- O sistema hexadecimal permite uma representação **mais compacta** dos números (com menos dígitos) que os sistemas decimal ou binário.

Conversão entre representações em base 2 e 16

Conversão de binário para hexadecimal: formar grupos de **4 bits** e converter para o respetivo dígito hexadecimal. [Nota: $16 = 2^4$]

De binário para hexadecimal

$\overline{1101}\,\overline{0101}_2 = D5_H$

$1011101001_2 = \overline{0010}\,\overline{1110}\,\overline{1001}_2 = 2E9_H$

$111010{,}0101_2 = \overline{0011}\,\overline{1010}{,}\overline{0101}_2 = 3A{,}5_H$

O processo também funciona no sentido inverso.

De hexadecimal para binário

$FF_H = \overline{1111}\,\overline{1111}_2 = 11111111_2$

$1AC_H = \overline{0001}\,\overline{1010}\,\overline{1100}_2 = 110101100_2$

$5F{,}2A_H = \overline{0101}\,\overline{1111}{,}\overline{0010}\,\overline{1010}_2 = 1011111{,}0010101_2$

1. Introdução à representação de informação
2. Representação em binário de números sem sinal
   - Representação binária
   - Conversão de base
   - O sistema de numeração hexadecimal
   - Operações aritméticas com operandos em binário
3. Representação de números inteiros com sinal
   - Operandos e resultado de tamanho fixo
   - Sinal e grandeza
   - Representação em complemento para 2
4. Representação de carateres e símbolos gráficos
5. Representação de números em vírgula flutuante
   - Conceitos gerais
   - Formato IEEE-754 precisão simples
   - Operações aritméticas com operandos normalizados

Adição de números sem sinal

- Números sem sinal (não-negativos): {0, 1, 2, 3, ...}
  - Regras para as operações aritméticas são *análogas* às de aritmética decimal.
  - Adição: 0 + 0 = 0 1 + 0 = 0 + 1 = 1
  - Adição: 1 + 1 = 10

    fica 0 na posição corrente e há transporte de 1 unidade para a posição seguinte
  - Adição: 1 + 1 + 1 = 11

    resultado igual a 1 e transporte de 1 unidade para a posição seguinte

Adição em binário

1	1		1	1	1			
	0	1	0	1	1	1	0	1
+	1	1	0	1	0	1	0	0
1	0	0	1	1	0	0	0	1

	1		
		9	3
+	2	1	2
	3	0	5

Multiplicação de números sem sinal

➠ Regras: $0 \times 0 = 1 \times 0 = 0 \times 1 = 0$ $1 \times 1 = 1$

Multiplicação em binário

$$
\begin{array}{cccccccc}
 & & & 1 & 0 & 1 & 0 & 0 \\
 & & \times & & 1 & 1 & 0 & 1 \\
\hline
 & & 1 & 1 & 0 & 1 & 0 & 0 \\
 & 1 & 0 & 0 & 0 & 0 & 0 & \\
1 & 1 & 1 & 0 & 1 & 0 & 0 & \\
+ & 1 & 0 & 1 & 0 & 0 & & \\
\hline
1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0
\end{array}
$$

$$
\begin{array}{ccccc}
 & & 2 & 0 \\
 & \times & 1 & 3 \\
\hline
 & & 6 & 0 \\
\times & 2 & 0 & \\
\hline
 & 2 & 6 & 0
\end{array}
$$

➠ É possível omitir as filas que têm apenas zeros, mas é preciso cuidado ao **alinhar os produtos parciais**.

Subtração de números sem sinal em binário

Importante: Subtrair **apenas** quando o diminuendo é maior ou igual ao subtraendo, isto é, calcular $a - b$ apenas se $a \geq b$.

- $1 - 1 = 0 - 0 = 0$ $\quad$ $1 - 0 = 1$ $\quad$ $0 - 1 =$ ✘
- Possível: $10 - 1 = 01$
  resultado = 1 e trocar o bit da posição seguinte (do diminuendo)
- Possível: $100 - 1 = 011$
  resultado=1 e trocar os bits das duas posições seguintes (do diminuendo)
- As alterações do diminuendo (em binário) seguem a regra:
  Da direita para a esquerda a partir da posição da subtração: trocar zeros por uns até ao primeiro 1; trocar também este dígito.

Subtração em binário

	0	0	1	0	1			
	1	1	0	1	0	0	1	1
-	0	1	0	1	1	1	1	0
	0	1	1	1	0	1	0	1

	1	0	
	2	1	1
-		9	4
	1	1	7

1. Introdução à representação de informação
2. Representação em binário de números sem sinal
   - Representação binária
   - Conversão de base
   - O sistema de numeração hexadecimal
   - Operações aritméticas com operandos em binário
3. Representação de números inteiros com sinal
   - Operandos e resultado de tamanho fixo
   - Sinal e grandeza
   - Representação em complemento para 2
4. Representação de carateres e símbolos gráficos
5. Representação de números em vírgula flutuante
   - Conceitos gerais
   - Formato IEEE-754 precisão simples
   - Operações aritméticas com operandos normalizados

Representações com número fixo de bits

- Processadores usam um **número fixo de bits** para representar números. Um processador de 32 bits tem esta designação por (entre outras coisas) representar preferencialmente números com 32 bits.
- Muitos processadores suportam representações de vários tamanhos. Ex.: processadores Intel suportam operandos de 8, 16, 32 ou 64 bits.
- Tipicamente, apenas operações com operandos e resultado *do mesmo tamanho* (i.e., igual número de bits) são suportadas.
- Tamanho fixo implica que *nem todos os resultados são representáveis*.
- A gama de números sem sinal representáveis usando N bits é $[\mathbf{0}; \mathbf{2^N - 1}]$.

Saída da gama de representação: erro de *overflow*

Com $N = 8$, a gama da representação binária é $[0; 255]$.

O resultado da adição $230_{10} + 72_{10}$ não pode ser representado, embora os operandos possam.

Números sem sinal: deteção de overflow

- Como detetar que ocorre *overflow*?

Adição em binário com *overflow*

Considerar N=8.

	1	1		1	1	1		
		0	1	0	1	1	1	0	1
+	1	1	0	1	0	1	0	0
	~~1~~	0	0	1	1	0	0	0	1

		9	3
+	2	1	2
		4	9

Erro! *Overflow*
Correto: 305;
resultado:
305 − 256 = 49.

- Verificação do resultado é *necessária* nas operações com número fixo de bits.

- Regra: na operação de adição, a ocorrência de um transporte (1) para lá do bit mais significativo indica que o resultado está errado devido a *overflow*.

1. Introdução à representação de informação
2. Representação em binário de números sem sinal
   - Representação binária
   - Conversão de base
   - O sistema de numeração hexadecimal
   - Operações aritméticas com operandos em binário
3. Representação de números inteiros com sinal
   - Operandos e resultado de tamanho fixo
   - Sinal e grandeza
   - Representação em complemento para 2
4. Representação de carateres e símbolos gráficos
5. Representação de números em vírgula flutuante
   - Conceitos gerais
   - Formato IEEE-754 precisão simples
   - Operações aritméticas com operandos normalizados

Representação em sinal e grandeza (magnitude)

- A representação em sinal e grandeza com N bits usa o bit mais significativo para indicar o sinal e os restantes $N - 1$ bits para representar o valor absoluto (a grandeza ou magnitude).
- Bit de sinal: **0 positivo, 1 negativo.**

Representação em sinal e grandeza

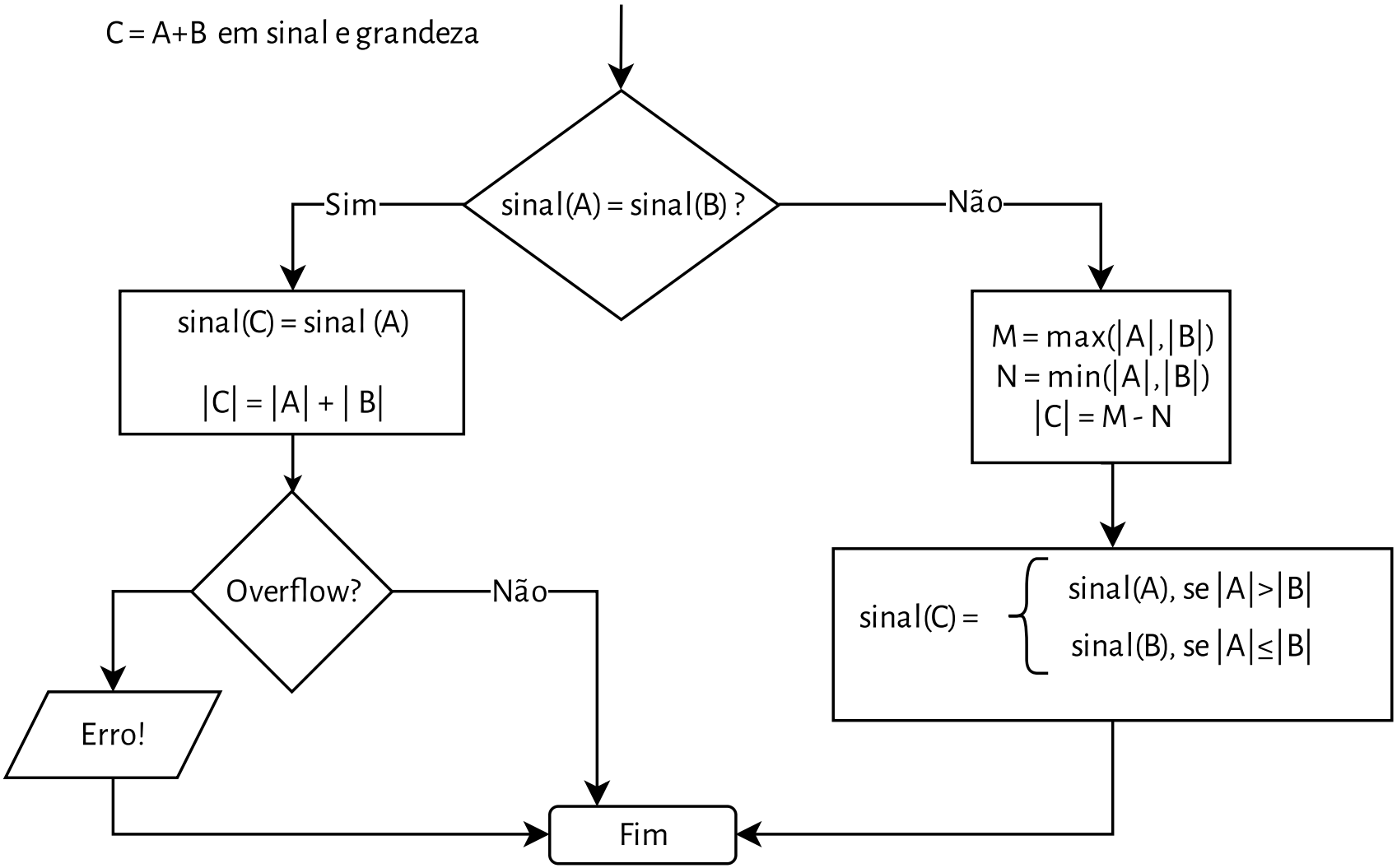
Considerar N=6.

$17_{10} = 010001_{SG}$ $\quad -17_{10} = 110001_{SG}$

A gama de valores representáveis é $[-31; +31]$.

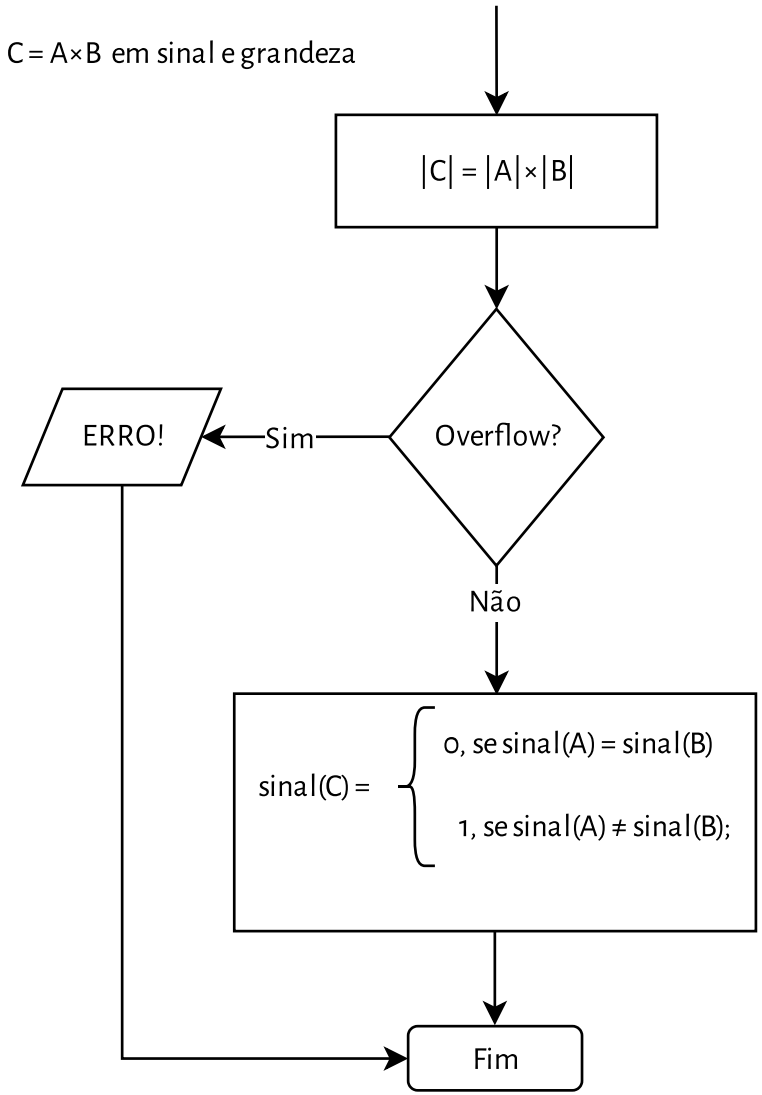
- A gama de valores representáveis com N bits é $[-(\mathbf{2^{N-1} - 1}); (\mathbf{2^{N-1} - 1})]$.
- Existem **duas** representações para 0.
- É uma representação muito pouco usada para números inteiros; não é suportada pela maioria dos processadores.

Regras para operações aritméticas: adição e subtração



➠ Subtração: transformar $A - B = A + (-B)$ e usar as regras da adição

Regras para operações aritméticas: multiplicação



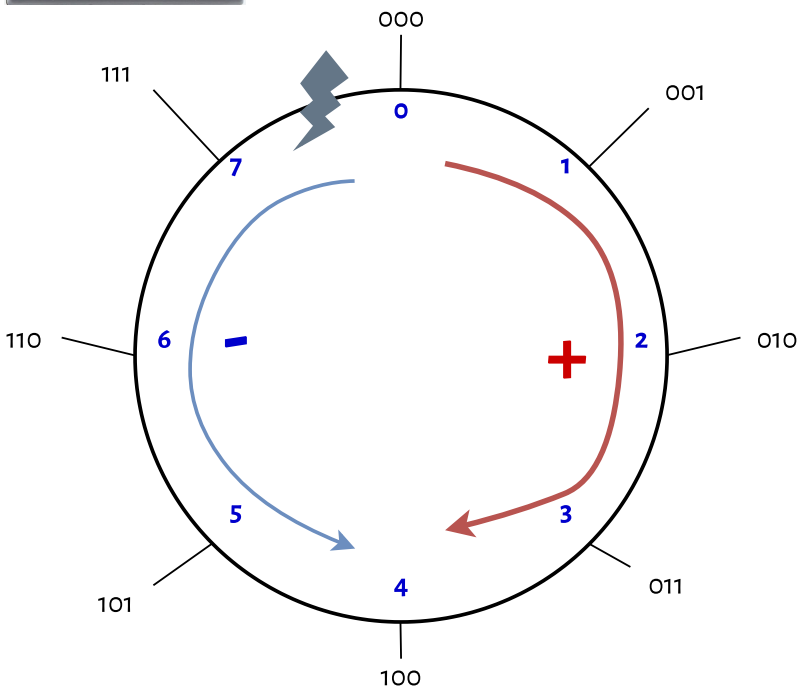
1. Introdução à representação de informação
2. Representação em binário de números sem sinal
   - Representação binária
   - Conversão de base
   - O sistema de numeração hexadecimal
   - Operações aritméticas com operandos em binário
3. Representação de números inteiros com sinal
   - Operandos e resultado de tamanho fixo
   - Sinal e grandeza
   - Representação em complemento para 2
4. Representação de carateres e símbolos gráficos
5. Representação de números em vírgula flutuante
   - Conceitos gerais
   - Formato IEEE-754 precisão simples
   - Operações aritméticas com operandos normalizados

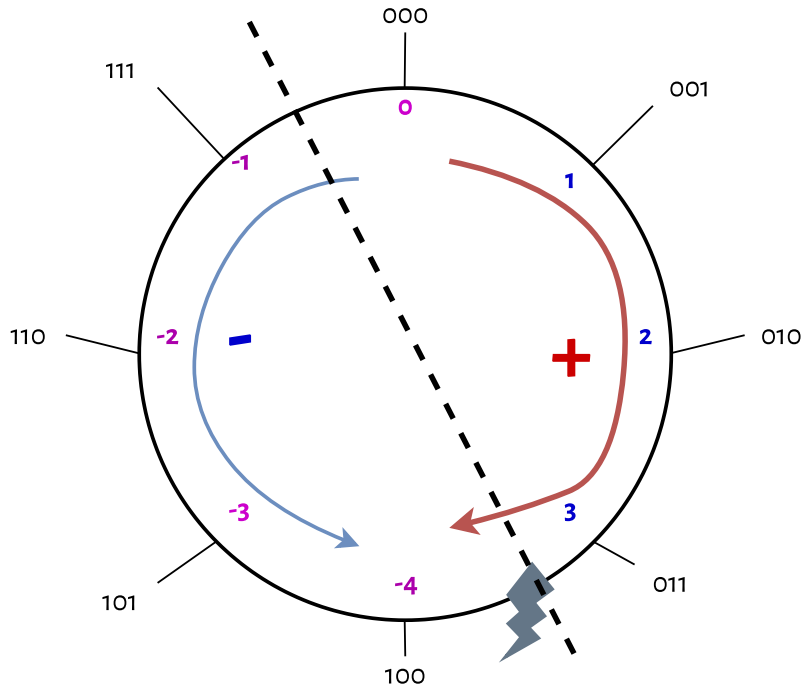
Representação em complemento para 2: conceito

- Objetivo: encontrar uma representação com regras de cálculo mais simples.
- O que acontece a um contador quando atinge o limite?



Valor vai a zero e a contagem recomeça.





- Intervalo de valores da representação em *complemento para 2*: **[-4; 3]**

Representação em complemento para 2: definição

- Representação em *complemento para 2* com **N** bits: $[-\mathbf{2}^{(\mathbf{N}-\mathbf{1})}; \mathbf{2}^{(\mathbf{N}-\mathbf{1})} - \mathbf{1}]$.
- Números **não-negativos** são representados (como números sem sinal)
  - ➠ Portanto, *bit mais significativo é zero*.
- Números **negativos** são representados de forma a satisfazer a fórmula:

$$-d_{N-1} \times 2^{N-1} + d_{N-2} \times 2^{N-2} + \ldots + d_1 \times 2^1 + d_0 \times 2^0$$

  - ➠ Implica que *bit mais significativo é um*.

Conversão de números com sinal (CPL2)

Usando N=5, o intervalo de valores representáveis é [-16;15].

$13_{10} = 01101_{cpl2}$

$-13_{10} = -16 + 3 = 10011_{cpl2}$

Representação em complemento para 2: operações

- Adição e subtração: procedimento é igual ao usado com números sem sinal, mas a deteção de *overflow* é diferente.
- Alternativa para subtração: calcular $A - B = A + (-B)$
- Ocorre *overflow* na **adição** de dois números em CPL2 **se e só se**:
  1. os dois operandos tiverem o mesmo sinal **e**
  2. o sinal do resultado for diferente do sinal dos operandos
- **Ignora-se** sempre o transporte para lá do dígito mais significativo.
- Para determinar o simétrico (trocar o sinal) de x;
  1. calcular em binário $(2^N - x)$ e ficar com os N bits menos significativos;
  2. **equivalente:** trocar todos os bits de x e somar 1;
  3. **regra prática:** da direita para a esquerda, copiar os dígitos de x até ao 1º um; depois inverter os dígitos restantes.

Representação em complemento para 2: exemplos

Exemplo: $N = 6$, intervalo $[-32; 31]$.

Cálculo do simétrico

Seja $x = -21_{10}$. Então, $-x = 21_{10} = 010101_{cpl2}$.

1. Como $2^6 = 1000000_2$,

	1	0	0	0	0	0	0
-		0	1	0	1	0	1
	~~0~~	1	0	1	0	1	1

2. $010101 \rightarrow 101010 + 1 = 101011$

3. $010101 \rightarrow 101011$

Logo $x = -21_{10} = 101011_{cpl2}$

Adição e subtração

Seja $y = -13_{10}$. Converter para CPL2: $-y = 13_{10} = 001101_{cpl2} \rightarrow y = 110011_{cpl2}$.

$x + y = 101011 + 110011 = \cancel{1}011110 \rightarrow$ *overflow!* (sinal do resultado diferente do sinal dos operandos)

$x - y = x + (-y) = 101011 + 001101 = 111000 \rightarrow$ *OK!*

Extensão do formato de representação

- Com dados representados em N bits, pode surgir a necessidade de obter a representação do mesmo valor em M bits.
- Em geral, é necessário ter $M > N$ para não haver perda de informação.
- As regras são as seguintes:

Formato	Regra	Exemplo (N=4, M=6)
sem sinal	acrescentar zeros à esquerda	1010 $\longrightarrow$ 001010
sinal e grandeza	acrescentar zeros à direita do bit de sinal	1010 $\longrightarrow$ 100010
complemento para 2	replicar o bit de sinal	**1**010 $\longrightarrow$ 111010 **0**110 $\longrightarrow$ 000110

Multiplicação em complemento para 2

- O produto de dois números de *N* bits pode requerer **2N** bits para uma representação correta.
- O algoritmo de multiplicação para números sem sinal aplicado a números em complemento para 2 não dá sempre o resultado correto.
- Complemento para 2: estender os produtos parciais para 2*N* bits de acordo com a seguinte tabela (op1 × op2):

Operando 1	Operando 2	Produtos parciais
+	+	sem alteração; extender o resultado
-	+	extensão de sinal para 2N bits
outros casos		trocar sinais dos operandos e aplicar regras anteriores

Multiplicação em complemento para 2: exemplo

Exemplo: Com N=5, a gama de representação é [-16; 15].

Multiplicação em complemento para dois (N=5)

- Com $a = -4_{10} = 11100_{cpl2}$ e $b = 3_{10} = 00011_{cpl2}$
- o resultado é $a \times b = -12_{10} = 1111110100_{cpl2}$.

Usar 10 bits do resultado.

						1	1	1	0	0
					×	0	0	0	1	1
	1	1	1	1	1	1	1	1	0	0
1	1	1	1	1	1	1	1	0	0	
/	1	1	1	1	1	1	0	1	0	0

- Neste caso, o resultado final cabe na gama de representação:

  Reduzir para 5 bits: $1111110100_{cpl2} \Rightarrow 10100_{cpl2}$

1 Introdução à representação de informação

2 Representação em binário de números sem sinal
- Representação binária
- Conversão de base
- O sistema de numeração hexadecimal
- Operações aritméticas com operandos em binário

3 Representação de números inteiros com sinal
- Operandos e resultado de tamanho fixo
- Sinal e grandeza
- Representação em complemento para 2

4 Representação de carateres e símbolos gráficos

5 Representação de números em vírgula flutuante
- Conceitos gerais
- Formato IEEE-754 precisão simples
- Operações aritméticas com operandos normalizados

Máquina de escrever

Máquina com carreto, alavanca de retorno e mecanismo de avanço de linha.



Fonte: `https://commons.wikimedia.org/wiki/File:Erika_9_typewriter.jpg`

Teletypewriter

Posto de introdução de dados / impressão *(teletypewriter)*



Marcin Wichary, CC BY 2.0, via Wikimedia Commons

Código ASCII

- American Standard Code for Information Interchange

Dec	Hex	Oct	Chr	Dec	Hex	Oct	HTML	Chr	Dec	Hex	Oct	HTML	Chr	Dec	Hex	Oct	HTML	Chr
0	0	000	NULL	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SoH	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	SoTxt	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	EoTxt	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EoT	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	Enq	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	Ack	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	Bell	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	Bsp	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	HTab	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LFeed	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VTab	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FFeed	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SOut	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SIn	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAck	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	Syn	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	EoTB	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	Can	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EoM	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	Sub	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	Esc	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FSep	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GSep	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RSep	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	USep	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	Delete

charstable.com

Código ISO-8859-1 (Latin-1)

ISO/IEC 8859-1:1998, Information technology — Part 1: Latin alphabet No. 1

ISO-8859-1																
	x0	**x1**	**x2**	**x3**	**x4**	**x5**	**x6**	**x7**	**x8**	**x9**	**xA**	**xB**	**xC**	**xD**	**xE**	**xF**
0x	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1x	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2x	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7x	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL
8x	PAD	HOP	BPH	NBH	IND	NEL	SSA	ESA	HTS	HTJ	VTS	PLD	PLU	RI	SS2	SS3
9x	DCS	PU1	PU2	STS	CCH	MW	SPA	EPA	SOS	SGCI	SCI	CSI	ST	OSC	PM	APC
Ax	NBSP	¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	-	®	¯
Bx	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
Cx	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
Dx	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
Ex	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
Fx	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Fonte: `https://www.alsacreations.com/`

Unicode

- A norma Unicode define uma sequência de «code points» (números inteiros)
  - Gama de 0 a 1114111: apenas 1112064 «code points» podem ser definidos.
  - Pontos representados em hexadecimal: `U+0000` – `U+10FFFF`.
- Os pontos de código podem ser codificados de várias maneiras, entre as quais:
  - UTF-8: a cada ponto corresponde uma sequência de 1 a 4 octetos;
  - UTF-32: a cada ponto de código corresponde um número inteiro de 32 bits.

π Я 音 æ∞

Símbolo	Unicode	UTF-32	UTF-8
Pi minúsculo	`U+03C0`	`0x000003C0`	`0xCF 0x80`
Ya maiúsculo (cirílico)	`U+042F`	`0x0000042F`	`0xD0 0xAF`
CJK: "som/música"	`U+97F3`	`0x000097F3`	`0xE9 0x9F 0xB3`
AE latino minúsculo	`U+00E6`	`0x000000E6`	`0xC3 0xA6`
Símbolo de infinito	`U+221E`	`0x0000221E`	`0xE2 0x88 0x9E`

1 Introdução à representação de informação

2 Representação em binário de números sem sinal
- Representação binária
- Conversão de base
- O sistema de numeração hexadecimal
- Operações aritméticas com operandos em binário

3 Representação de números inteiros com sinal
- Operandos e resultado de tamanho fixo
- Sinal e grandeza
- Representação em complemento para 2

4 Representação de carateres e símbolos gráficos

5 Representação de números em vírgula flutuante
- Conceitos gerais
- Formato IEEE-754 precisão simples
- Operações aritméticas com operandos normalizados

Notação científica

- Em muitas situações é preciso representar números de uma gama alargada, desde números muito próximos de zero até números de grande magnitude: cálculo científico, contabilidade, computação gráfica, etc.
- A chamada *notação científica* é o formato mais usado.
- Nesta notação em decimal, os números têm o formato $d_0,d_{-1}d_{-2}d_{-3}\ldots \times 10^{i}$ em que i é um número inteiro.

Exemplos

$17{,}987 = 1{,}7987 \times 10^{1}$

$0{,}0004567 = 4{,}567 \times 10^{-4}$

$1{,}756 = 175{,}6 \times 10^{-2} = 0{,}001756 \times 10^{3} = \ldots$

- Para obter uma representação **única**, usa-se uma condição de *normalização*.
  Exemplo: os números devem ter apenas 1 dígito ($\neq 0$) à esquerda da vírgula.
  $4{,}567 \times 10^{-4}$ ✔ $\quad$ $0{,}0004567$ ✘ $\quad$ $456{,}7 \times 10^{-6}$ ✘

Norma para representação em vírgula flutuante

- A representação correspondente à notação científica é designada por representação em *vírgula flutuante* (“floating point”).
- A parte $d_0,d_{-1}d_{-2}d_{-3}\ldots$ é designada por *significando* (tradicionalmente, a parte fracionária é a *mantissa*).
- Os formatos usados atualmente são definidos por uma norma internacional ISO/IEC/IEEE 60559:2020 (também designada norma IEEE-754).
- A norma define várias representações de precisão diferente.
- O formato de **precisão simples** usa 32 bits; o de **precisão dupla**, 64 bits.
- A normas inclui uma representação não normalizada para números próximos de zero (números *denormais*).
- Define vários modos de arredondamento.
- Inclui representações para conceitos não-numéricos: infinito, erros, etc.

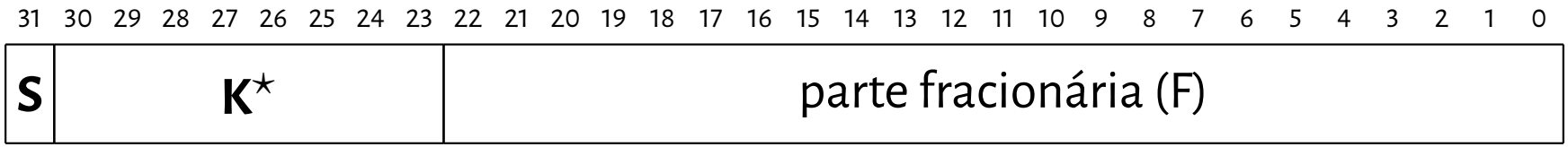
Definição do formato VF de precisão simples

- O formato IEEE-754 codifica números representados **em binário** na seguinte forma *normalizada*:

$$\pm \mathbf{1},\mathbf{F} \times \mathbf{2}^{\mathbf{K}}$$

  - **F** parte fracionária do significando normalizada 1,.. (em binário)
  - **K** expoente inteiro na gama [-126;+127]

- A organização do formato é a seguinte:



  - **S** 0 se positivo, 1 se negativo (sinal)
  - **K*** $\mathbf{K}^\star = \mathbf{K} + \mathbf{127}$ (em binário) — codificação *excesso-127*
    Portanto, $K^\star \in [1; 254]$

- Generalização: Para um expoente de **E** bits usa-se a notação excesso $\mathbf{2}^{(\mathbf{E}-\mathbf{1})} - \mathbf{1}$

Conversão decimal $\leftrightarrow$ VF

Conversão decimal $\rightarrow$ VF

X=−12,25 em binário $|X| = 1100{,}01_2 = 1{,}10001_2 \times 2^3$

$K = 3 \rightarrow K^\star = 127 + 3 = 130 = 10000010_2$

31	30 29 28 27 26 25 24	23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
1	10000010	10001000000000000000000

1100 0001 0100 0100 0000 0000 0000 0000 $\rightarrow$ C1440000$_H$

Conversão VF $\rightarrow$ decimal

Y é representado em VF pelo padrão 41B90000$_H$. Valor decimal?

41B90000$_H$ $\rightarrow$ 0100 0001 1011 1001 0000 0000 0000 0000

0	10000011	01110010000000000000000

$K^\star = 10000011_2 = 131 \rightarrow K = 131 - 127 = 4$

$|Y| = 1{,}0111001_2 \times 2^4 = 10111{,}001_2 = 23{,}125 \rightarrow Y = 23{,}125$ porque $S = 0$

Casos especiais

- $\mathbf{K^\star = 255}$ Representação de situações excecionais (erro, infinito, etc.).
- $\mathbf{K^\star = 0}$ Representação não normalizada *(denormal)*.

  Neste caso, o número representado é

  $$\pm 0{,}F \times 2^{-126}$$

  Exemplo: $\mathbf{2^{-127}} = 0{,}1 \times \mathbf{2^{-126}}$ é um número subnormal
  com $F = 10000000000000000000000_2$ (são 22 zeros)

  O bit à esquerda da vírgula é (implicitamente) 0.

- O número zero também é denormal: $F = 0$, $K^\star = 0$.
- O uso de números subnormais permite **uma aproximação mais gradual a zero**, mas têm uma precisão inferior à dos números normalizados.

Regras para adição / subtração

- O formato VF é uma representação em *sinal e grandeza*.
- Calcular $C = A + B$ usando a representação em vírgula flutuante
  1. Comparar $|A|$ e $|B|$: X=max($|A|$,$|B|$) e Y=min($|A|$,$|B|$)
  2. Alterar a mantissa de Y para este ter o mesmo expoente que X.
  3. Se sinal(A)=sinal(B)
     1. $|C| = X + Y$
     2. sinal(C) = sinal(A)
  4. Se sinal(A) $\neq$ sinal(B)
     1. $|C| = X - Y$
     2. sinal(C)=sinal do operando de maior valor absoluto
  5. Normalizar C (se necessário)
- Subtração é transformada em adição: $C = A - B = A + (-B)$

Regras para a multiplicação

- Calcular $C = A \times B$ em VF
  1. Se sinal(A)=sinal(B) então sinal(C)=0 senão sinal(C)=1
  2. Expoente(C)=Expoente(A)+Expoente(B)
  3. Mantissa(C)=Mantissa(A)$\times$Mantissa(B)
  4. Normalizar C (se necessário)
- Atenção: a soma de dois operandos codificados em *excesso-127* faz-se somando os dois operandos normalmente e subtraindo 127.

$$E_C = E_A + E_B - 127$$

em que E_A, E_B e E_C representam os expoentes **codificados**.

Outros formatos com vírgula flutuante: IEEE binary16

- Formato mais curto.

Formato de 16 bits (IEEE binary16)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	1	1	0	0	0	0	0	0	0	1
S	expoente (5 bits)					fração (10 bits)									

Valor: $-1.1100000001_2 \times 2^{17-15} = -111.00000001_2 = -\left(7 + 2^{-8}\right) = -7{,}00390625$

- Mesmas regras que representação de precisão simples (binary32), mas com parâmetros diferentes.
- Viés do expoente: 15
- Valor máximo: aproximadamente $6{,}55 \times 10^4$
- Cerca de 3 dígitos decimais de precisão
- Usos típicos: gráficos e aprendizagem computacional.

Outros formatos com vírgula flutuante: bfloat16

- Formato «brain float» usado pela Google no **treino** de sistemas de aprendizagem computacional.
- Suportado por muitos processadores recentes.

Brain Float 16 bits (bfloat16)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	1	1	0	0	0	0	0	0	0	1
S	expoente (8 bits)								fração (7 bits)						

Valor: $-1.0000001_2 \times 2^{142-127} = -\left(1 + 2^{-7}\right) \times 2^{15} = -33024$

- Parecido com precisão simples (IEEE binary32):
  Mesmo tamanho do expoente, aproximadamente a mesma gama, mesmo viés do expoente (127)
- Apenas 2-3 dígitos decimais de precisão.
- Maior gama que binary16, mas menor precisão.